

ANAS MOHAMMED KT

2025MSBDA029

Restaurant Data Analytics and Predictive Modeling

Data Exploration and Preprocessing

```
In [1]: import warnings

warnings.filterwarnings("ignore")
```

```
In [2]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns; sns.set(color_codes=True)
%matplotlib inline
```

```
In [3]: df = pd.read_csv("Dataset .csv")
```

Explore the dataset and identify the number of rows and columns

```
In [4]: df.head()
```

Out[4]:

	Restaurant ID	Restaurant Name	Country Code	City	Address	Locality	Locality Verbose
0	6317637	Le Petit Souffle	162	Makati City	Third Floor, Century City Mall, Kalayaan Avenu...	Century City Mall, Poblacion, Makati City	Century City Mall, Poblacion, Makati City, Mak...
1	6304287	Izakaya Kikufuji	162	Makati City	Little Tokyo, 2277 Chino Roces Avenue, Legaspi...	Little Tokyo, Legaspi Village, Makati City	Little Tokyo, Legaspi Village, Makati City, Ma...
2	6300002	Heat - Edsa Shangri-La	162	Mandaluyong City	Edsa Shangri-La, 1 Garden Way, Ortigas, Mandal...	Edsa Shangri-La, Ortigas, Mandaluyong City	Edsa Shangri-La, Ortigas, Mandaluyong City, Ma...
3	6318506	Ooma	162	Mandaluyong City	Third Floor, Mega Fashion Hall, SM Megamall, O...	SM Megamall, Ortigas, Mandaluyong City	SM Megamall, Ortigas, Mandaluyong City, Mandal...
4	6314302	Sambo Kojin	162	Mandaluyong City	Third Floor, Mega Atrium, SM Megamall, Ortigas...	SM Megamall, Ortigas, Mandaluyong City	SM Megamall, Ortigas, Mandaluyong City, Mandal...

5 rows × 21 columns



In [5]: df.info()

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9551 entries, 0 to 9550
Data columns (total 21 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Restaurant ID         9551 non-null   int64
 1   Restaurant Name       9551 non-null   object
 2   Country Code         9551 non-null   int64
 3   City                 9551 non-null   object
 4   Address              9551 non-null   object
 5   Locality             9551 non-null   object
 6   Locality Verbose     9551 non-null   object
 7   Longitude            9551 non-null   float64
 8   Latitude             9551 non-null   float64
 9   Cuisines              9542 non-null   object
10   Average Cost for two 9551 non-null   int64
11   Currency             9551 non-null   object
12   Has Table booking    9551 non-null   object
13   Has Online delivery  9551 non-null   object
14   Is delivering now    9551 non-null   object
15   Switch to order menu 9551 non-null   object
16   Price range          9551 non-null   int64
17   Aggregate rating     9551 non-null   float64
18   Rating color         9551 non-null   object
19   Rating text          9551 non-null   object
20   Votes               9551 non-null   int64
dtypes: float64(3), int64(5), object(13)
memory usage: 1.5+ MB

```

```
In [6]: df.shape
```

```
Out[6]: (9551, 21)
```

```
In [7]: df.isnull().sum()
```

```
Out[7]: Restaurant ID      0
        Restaurant Name   0
        Country Code     0
        City              0
        Address            0
        Locality           0
        Locality Verbose  0
        Longitude          0
        Latitude           0
        Cuisines           9
        Average Cost for two 0
        Currency           0
        Has Table booking  0
        Has Online delivery 0
        Is delivering now  0
        Switch to order menu 0
        Price range        0
        Aggregate rating   0
        Rating color       0
        Rating text        0
        Votes              0
        dtype: int64
```

```
In [8]: df["Cuisines"].fillna("Not specified", inplace=True)
```

Check for missing values in each column and handle them accordingly

```
In [9]: df.isnull().sum()
```

```
Out[9]: Restaurant ID      0
        Restaurant Name   0
        Country Code     0
        City              0
        Address            0
        Locality           0
        Locality Verbose  0
        Longitude          0
        Latitude           0
        Cuisines           0
        Average Cost for two 0
        Currency           0
        Has Table booking  0
        Has Online delivery 0
        Is delivering now  0
        Switch to order menu 0
        Price range        0
        Aggregate rating   0
        Rating color       0
        Rating text        0
        Votes              0
        dtype: int64
```

```
In [10]: dup = df.duplicated().sum()
         print(f'Number of Duplicated Rows are {dup}')
```

Number of Duplicated Rows are 0

Perform data type conversion if necessary. Analyze the distribution of the target variable("Aggregate rating") and identify any class imbalance

In [11]: `df.info()`

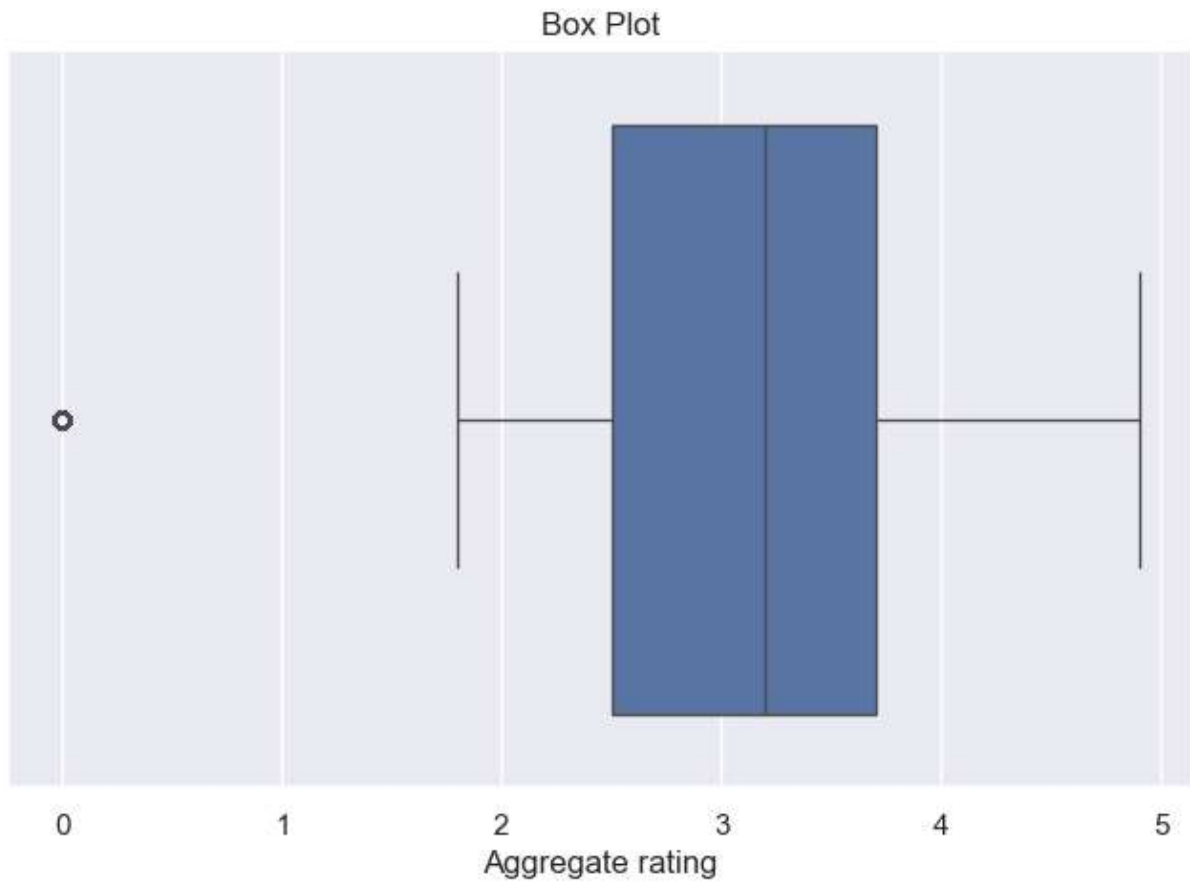
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9551 entries, 0 to 9550
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Restaurant ID          9551 non-null   int64
1   Restaurant Name        9551 non-null   object
2   Country Code           9551 non-null   int64
3   City                   9551 non-null   object
4   Address                9551 non-null   object
5   Locality               9551 non-null   object
6   Locality Verbose       9551 non-null   object
7   Longitude              9551 non-null   float64
8   Latitude               9551 non-null   float64
9   Cuisines               9551 non-null   object
10  Average Cost for two   9551 non-null   int64
11  Currency               9551 non-null   object
12  Has Table booking      9551 non-null   object
13  Has Online delivery    9551 non-null   object
14  Is delivering now      9551 non-null   object
15  Switch to order menu   9551 non-null   object
16  Price range            9551 non-null   int64
17  Aggregate rating       9551 non-null   float64
18  Rating color           9551 non-null   object
19  Rating text            9551 non-null   object
20  Votes                  9551 non-null   int64
dtypes: float64(3), int64(5), object(13)
memory usage: 1.5+ MB
```

In [12]: `target = "Aggregate rating"`

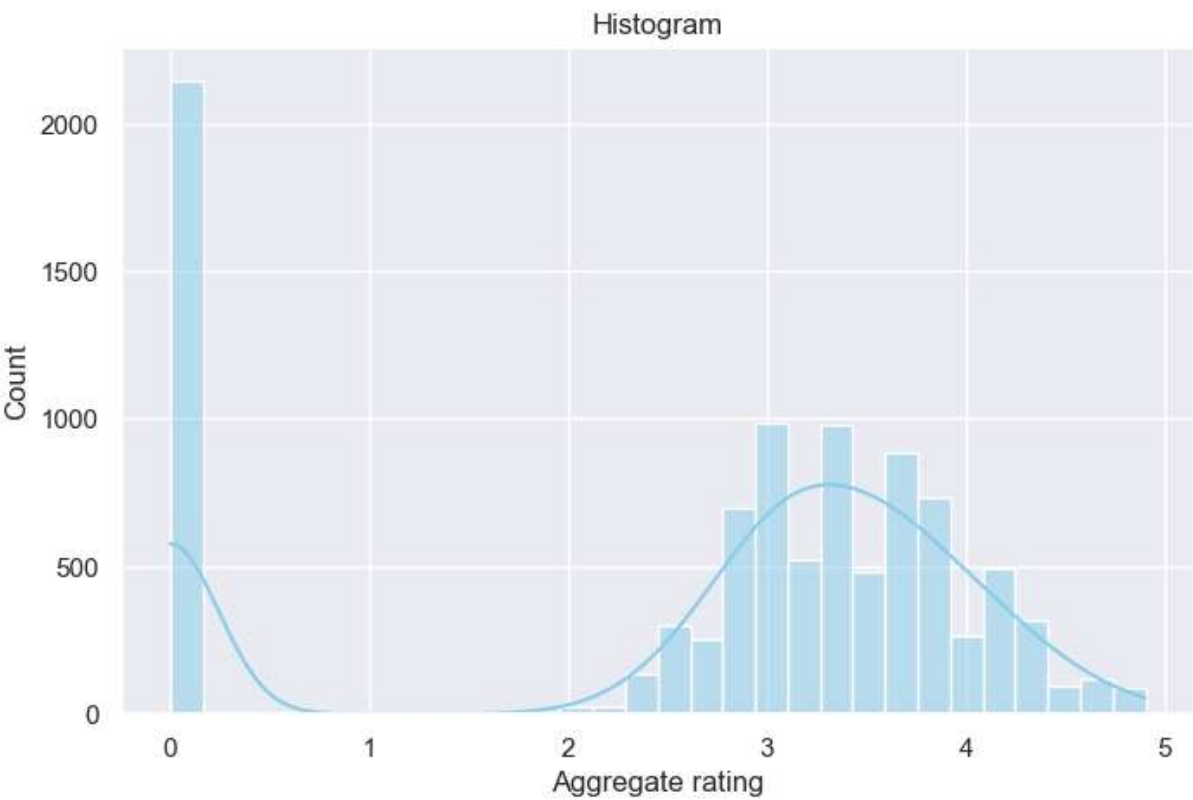
```
print(df[target].describe())
```

```
count    9551.000000
mean       2.666370
std        1.516378
min         0.000000
25%        2.500000
50%        3.200000
75%        3.700000
max         4.900000
Name: Aggregate rating, dtype: float64
```

In [13]: `plt.figure(figsize=(8,5))`
`sns.boxplot(x=df[target])`
`plt.title("Box Plot")`
`plt.xlabel("Aggregate rating")`
`plt.show()`



```
In [14]: plt.figure(figsize=(8,5))
sns.histplot(df[target],bins=30, kde=True, color="skyblue")
plt.title("Histogram")
plt.xlabel("Aggregate rating")
plt.show()
```



Calculate basic statistical measures(mean,median,standard deviation,etc.) for numerical columns

```
In [15]: df.describe()
```

Out[15]:

	Restaurant ID	Country Code	Longitude	Latitude	Average Cost for two	Price range	
count	9.551000e+03	9551.000000	9551.000000	9551.000000	9551.000000	9551.000000	9
mean	9.051128e+06	18.365616	64.126574	25.854381	1199.210763	1.804837	
std	8.791521e+06	56.750546	41.467058	11.007935	16121.183073	0.905609	
min	5.300000e+01	1.000000	-157.948486	-41.330428	0.000000	1.000000	
25%	3.019625e+05	1.000000	77.081343	28.478713	250.000000	1.000000	
50%	6.004089e+06	1.000000	77.191964	28.570469	400.000000	2.000000	
75%	1.835229e+07	1.000000	77.282006	28.642758	700.000000	2.000000	
max	1.850065e+07	216.000000	174.832089	55.976980	800000.000000	4.000000	

```
In [16]: df[["Average Cost for two","Price range","Aggregate rating","Votes"]].describe()
```

Out[16]:

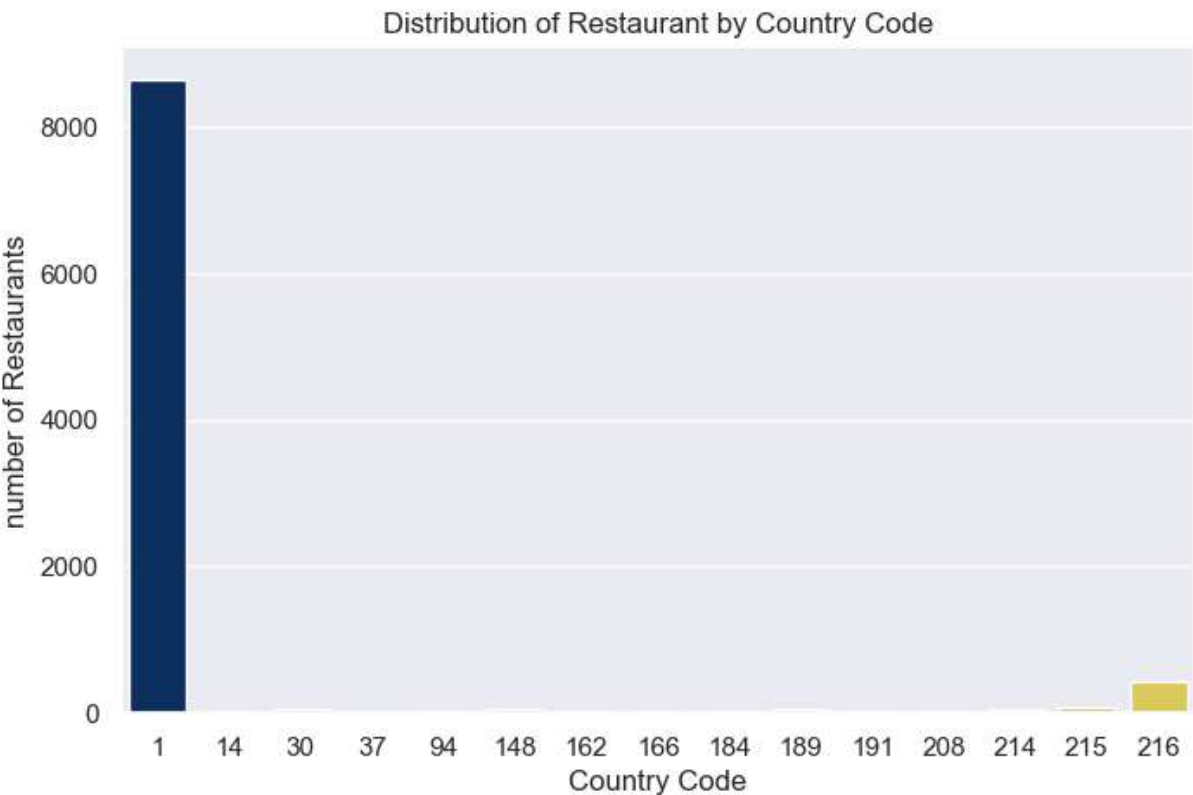
	Average Cost for two	Price range	Aggregate rating	Votes
count	9551.000000	9551.000000	9551.000000	9551.000000
mean	1199.210763	1.804837	2.666370	156.909748
std	16121.183073	0.905609	1.516378	430.169145
min	0.000000	1.000000	0.000000	0.000000
25%	250.000000	1.000000	2.500000	5.000000
50%	400.000000	2.000000	3.200000	31.000000
75%	700.000000	2.000000	3.700000	131.000000
max	800000.000000	4.000000	4.900000	10934.000000

Explore the distribution of categorical variables like "Country Code","City", and "Cuisines".Identify the top cuisines and cities with the highest number of restaurants.

In [17]:

```
plt.figure(figsize=(8,5))
sns.countplot(x="Country Code", data=df, palette="cividis")

plt.title("Distribution of Restaurant by Country Code")
plt.xlabel("Country Code")
plt.ylabel("number of Restaurants")
plt.show()
```



```
In [18]: top_countries = df["Country Code"].value_counts().head()

print("Top 5 Countries with highest number of Restaurants:")
print(top_countries)
```

Top 5 Countries with highest number of Restaurants:

Country Code

1 8652

216 434

215 80

30 60

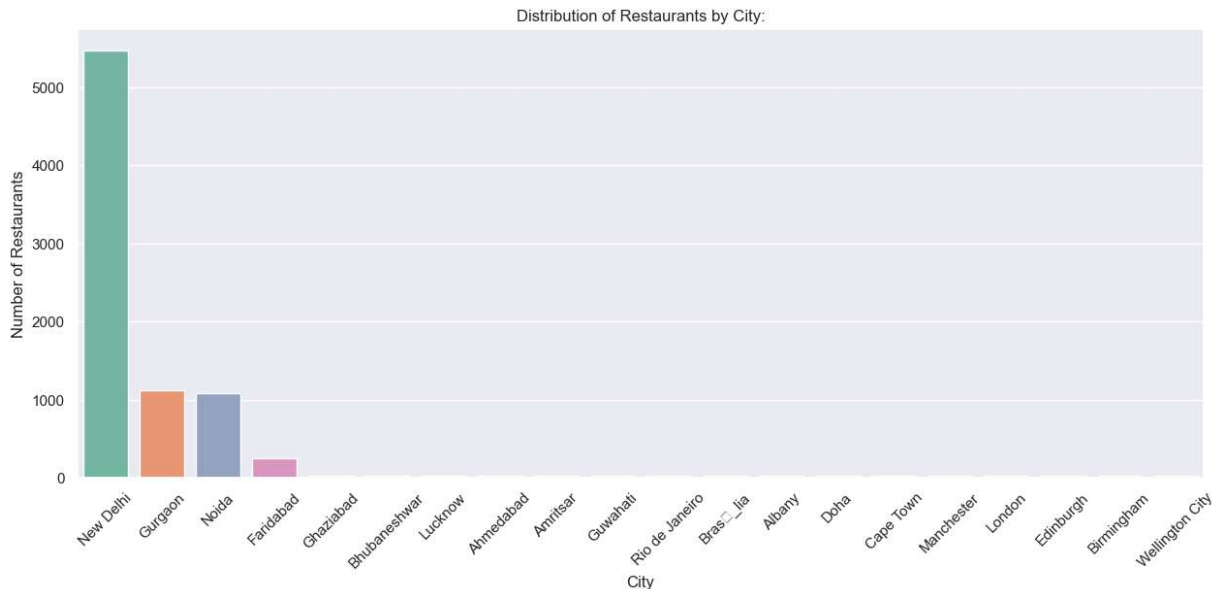
189 60

Name: count, dtype: int64

```
In [19]: plt.figure(figsize=(15,6))

sns.countplot(x="City", data=df, order=df["City"].value_counts().head(20).index, palette="Set2")

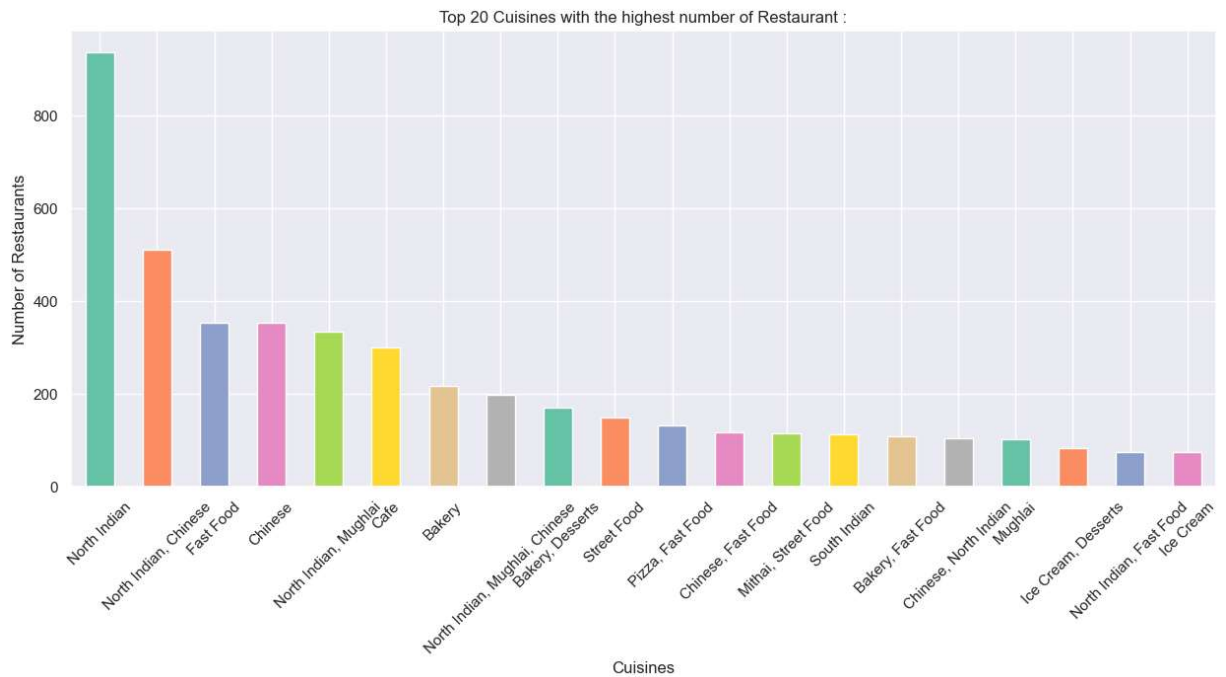
plt.title("Distribution of Restaurants by City:")
plt.xlabel("City")
plt.ylabel("Number of Restaurants")
plt.xticks(rotation=45)
plt.show()
```



```
In [20]: plt.figure(figsize=(15,6))

cuisines_count = df['Cuisines'].value_counts()
cuisines_count.head(20).plot(kind="bar", color=sns.color_palette("Set2"))

plt.title("Top 20 Cuisines with the highest number of Restaurant :")
plt.xlabel("Cuisines")
plt.ylabel("Number of Restaurants")
plt.xticks(rotation=45)
plt.show()
```



```
In [21]: top_cities = df["City"].value_counts().head(10)

print("Top 10 Cities with the Highest Number of Restaurants:")
print(top_cities)
```

Top 10 Cities with the Highest Number of Restaurants:

City	count
New Delhi	5473
Gurgaon	1118
Noida	1080
Faridabad	251
Ghaziabad	25
Bhubaneswar	21
Lucknow	21
Ahmedabad	21
Amritsar	21
Guwahati	21

Name: count, dtype: int64

```
In [22]: top_cuisines = cuisines_count.head(10)

print("Top 10 Cuisines with the Highest Number of Restaurants :")
print(top_cuisines)
```

Top 10 Cuisines with the Highest Number of Restaurants :

Cuisines	
North Indian	936
North Indian, Chinese	511
Fast Food	354
Chinese	354
North Indian, Mughlai	334
Cafe	299
Bakery	218
North Indian, Mughlai, Chinese	197
Bakery, Desserts	170
Street Food	149

Name: count, dtype: int64

Visualize the locations of restaurants on a map using latitude and longitude information.

```
In [23]: from shapely.geometry import Point
import geopandas as gpd
from geopandas import GeoDataFrame
```

```
In [24]: import geopandas as gpd
import matplotlib.pyplot as plt

gdf = gpd.GeoDataFrame(
    df,
    geometry=gpd.points_from_xy(df.Longitude, df.Latitude)
)

# Load world countries dataset
world = gpd.read_file(
    "https://naturalearth.s3.amazonaws.com/110m_cultural/ne_110m_admin_0_countries."
)

# Create the map
fig, ax = plt.subplots(figsize=(18, 12))

# Plot countries with different colors for each continent
world.plot(
    ax=ax,
    column="CONTINENT",
    categorical=True,
    legend=True,
    legend_kwds={
        "title": "Continent Colors",
        "loc": "lower left"
    },
    edgecolor="black",
    linewidth=0.3
)

gdf.plot(
    ax=ax,
    marker="o",
    color="red",
```

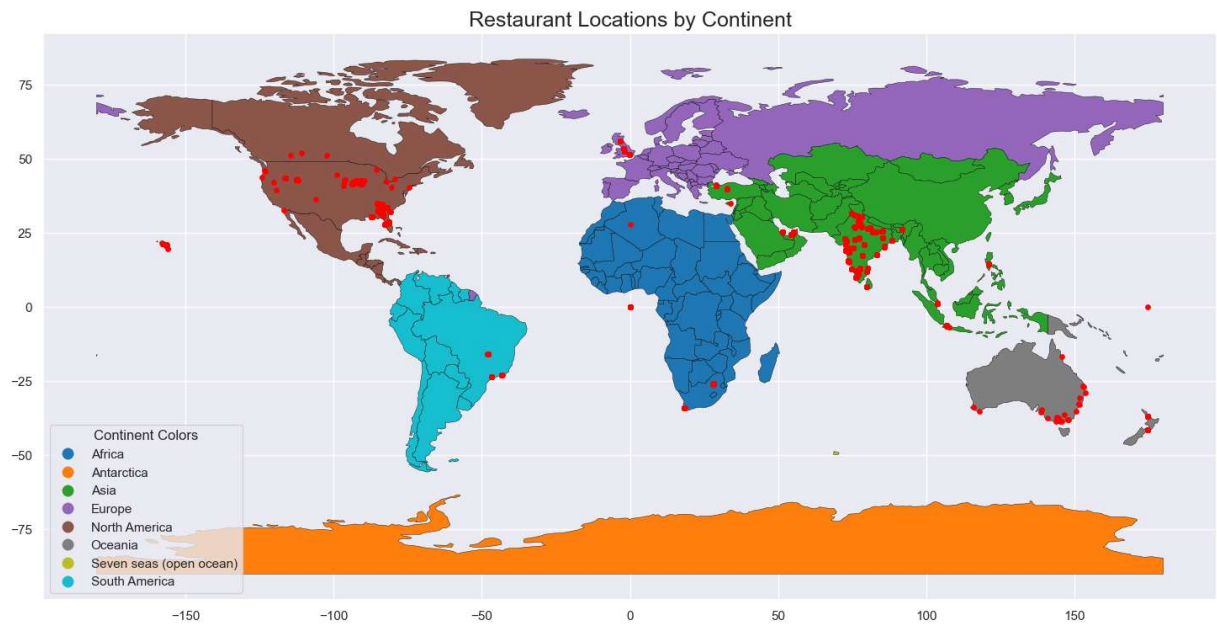
```

        markersize=15,
        label="Locations"
    )

    ax.set_title(
        "Restaurant Locations by Continent",
        fontsize=18
    )

    plt.show()

```

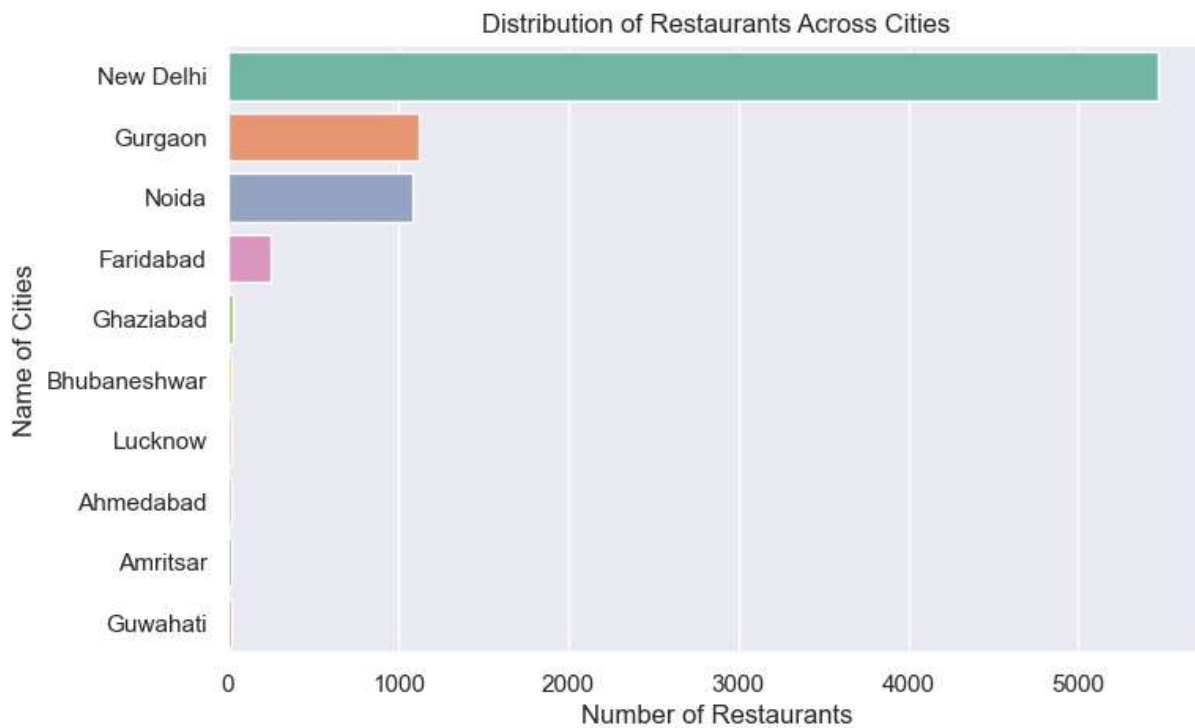


```

In [25]: plt.figure(figsize=(8,5))
sns.countplot(y=df["City"],order = df.City.value_counts().head(10).index, palette="

plt.xlabel("Number of Restaurants")
plt.ylabel("Name of Cities")
plt.title("Distribution of Restaurants Across Cities")
plt.show()

```



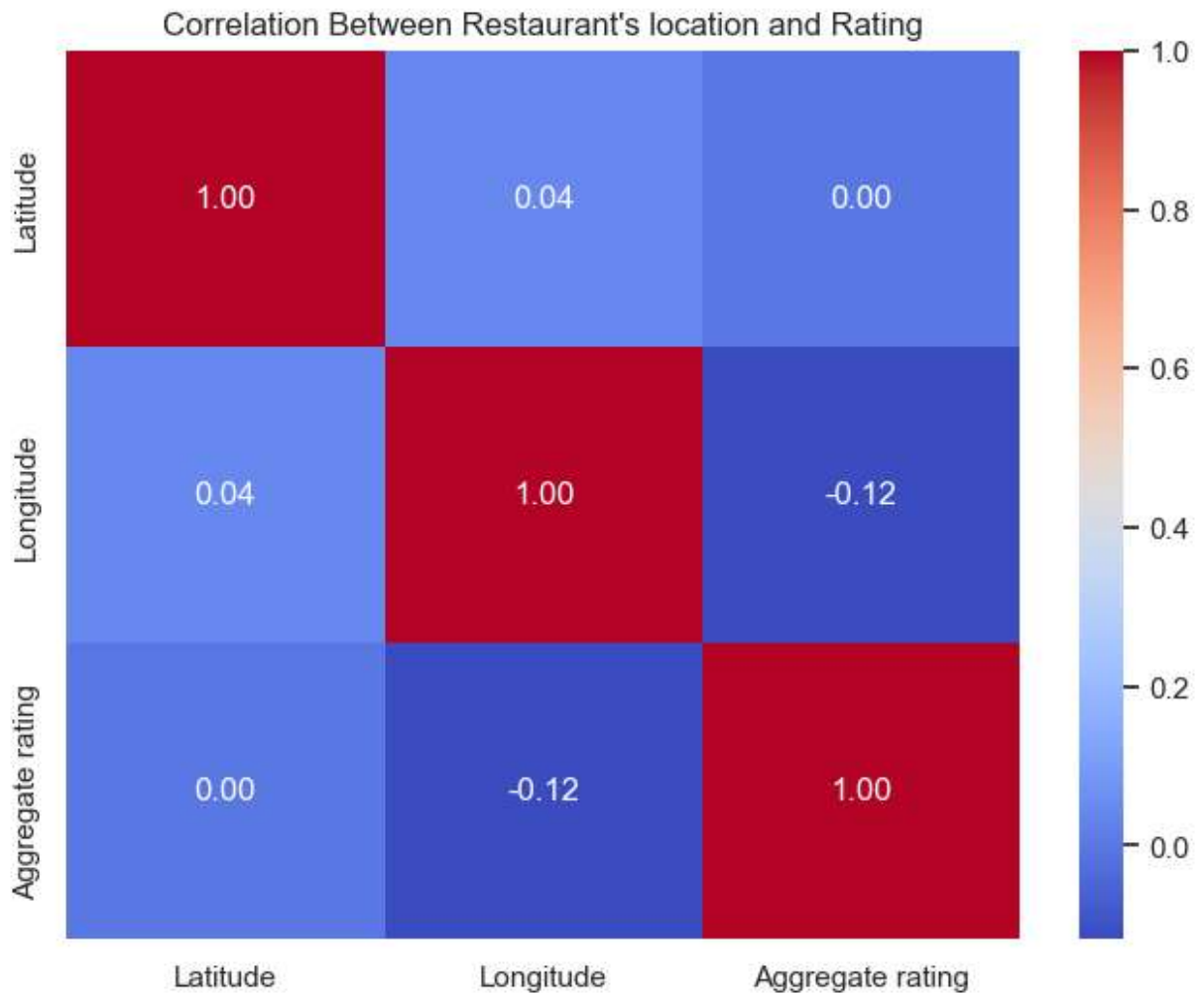
```
In [26]: plt.figure(figsize=(8,6))

correlation_matrix = df[["Latitude","Longitude","Aggregate rating"]].corr()

sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm",fmt=".2f")

plt.title("Correlation Between Restaurant's location and Rating")

plt.show()
```



Determine the percentage of restaurants that offer table booking and online delivery

```
In [27]: df["Has Table booking"].value_counts()
```

```
Out[27]: Has Table booking
No      8393
Yes     1158
Name: count, dtype: int64
```

```
In [28]: df["Has Online delivery"].value_counts()
```

```
Out[28]: Has Online delivery
No      7100
Yes     2451
Name: count, dtype: int64
```

```
In [29]: print("Table Booking:",round((1158/(8393+1158)) *100, 2),"%")
print("Online Delivery:",round((2451/(7100+2451)) *100, 2), "%")
```

```
Table Booking: 12.12 %
Online Delivery: 25.66 %
```

```
In [30]: table_booking = df["Has Table booking"].value_counts().get("Yes")
online_delivery = df["Has Online delivery"].value_counts().get("Yes")
```

```
percentage_table_booking = (table_booking / len(df)) *100
percentage_online_delivery = (online_delivery / len(df)) *100

print(f"Percentage of Restaurant offers table booking: {percentage_table_booking: .")
print(f"Percentage of Restaurant offers online delivery : {percentage_online_delive
```

Percentage of Restaurant offers table booking: 12.12 %

Percentage of Restaurant offers online delivery : 25.66 %

Compare the average rating of restaurants with table booking and those without

```
In [31]: df_with_table_booking = df[df["Has Table booking"] == "Yes"]
df_without_table_booking = df[df["Has Table booking"] == "No"]

print("Rows with Table: " , df_with_table_booking.shape)
print("Rows without Table: ", df_without_table_booking.shape)
```

Rows with Table: (1158, 21)
Rows without Table: (8393, 21)

```
In [32]: print("Average Ratings: ")
print("With Table Booking : ",round(df_with_table_booking["Aggregate rating"].mean(
print("Without Table Booking : ",round(df_without_table_booking["Aggregate rating"]
```

Average Ratings:
With Table Booking : 3.44
Without Table Booking : 2.56

Analyze the availability of online delivery among restaurants with different price ranges

```
In [33]: Online_Delivery_by_price_range = df.groupby("Price range")["Has Online delivery"].v
Online_Delivery_by_price_range.plot(kind = "bar", stacked=True,colormap="viridis",f

plt.title("Online Delivery Availability by Price Range")
plt.xlabel("Price Range")
plt.ylabel("Percentage of Restaurants with Online Delivery")

plt.xticks(rotation = 0)
plt.legend(title= "Online Delivery",bbox_to_anchor=(1.05,1))
plt.show()
```



```
In [34]: Online_Delivery_Yes = df[df["Has Online delivery"] == "Yes"]
Online_Delivery_counts = Online_Delivery_Yes.groupby(["Price range", "Has Online del
Online_Delivery_counts.plot(kind="bar", stacked=True, colormap="cividis", figsize=(10,

plt.title("Online Delivery Availability by Price Range")
plt.xlabel("Price Range")
plt.ylabel("Number of Restaurants")
plt.xticks(rotation = 0)
plt.legend(title="Online Delivery", bbox_to_anchor=(1.05, 1), loc="upper left")
plt.show()
```



Determine the most common price range among all the restaurants

```
In [35]: df["Price range"].value_counts()
```

```
Out[35]: Price range
1      4444
2      3113
3      1408
4       586
Name: count, dtype: int64
```

```
In [36]: most_common=df["Price range"].mode()[0]
print("Most Common Price range among all the restaurant: ",most_common)
```

Most Common Price range among all the restaurant: 1

Calculate the average rating for each price range. Identify the color that represents the highest average rating among different price range

```
In [37]: Avg_Rating_by_price_range = df.groupby("Price range")["Aggregate rating"].mean().ro
print("Average Rating for each price range:")
print(Avg_Rating_by_price_range)
```

Average Rating for each price range:

Price range	Average Rating
1	2.00
2	2.94
3	3.68
4	3.82

Name: Aggregate rating, dtype: float64

```
In [38]: highest_avg_rating_color = Avg_Rating_by_price_range.idxmax()

plt.bar(Avg_Rating_by_price_range.index,Avg_Rating_by_price_range,color="skyblue",w
plt.bar(highest_avg_rating_color, Avg_Rating_by_price_range[highest_avg_rating_colo

plt.xlabel("Price Range")
plt.ylabel("Average Rating")
plt.title("Average Rating by Price Range")
plt.show()
```



Extract additional features from the existing columns, such as the length of the restaurant name or address

```
In [39]: df["Restaurant Name Length"] = df["Restaurant Name"].apply(lambda x: len(str(x)))  
df["Address Length"] = df["Address"].apply(lambda x: len(str(x)))
```

```
In [40]: df[["Restaurant Name", "Restaurant Name Length", "Address", "Address Length"]]
```

Out[40]:

	Restaurant Name	Restaurant Name Length	Address	Address Length
0	Le Petit Souffle	16	Third Floor, Century City Mall, Kalayaan Avenu...	71
1	Izakaya Kikufuji	16	Little Tokyo, 2277 Chino Roces Avenue, Legaspi...	67
2	Heat - Edsa Shangri-La	22	Edsa Shangri-La, 1 Garden Way, Ortigas, Mandal...	56
3	Ooma	4	Third Floor, Mega Fashion Hall, SM Megamall, O...	70
4	Sambo Kojin	11	Third Floor, Mega Atrium, SM Megamall, Ortigas...	64
...	...	...	...	...
9546	Namll Gurme	11	Kemanke Karamustafa Pa Mahallesi, Rlhtlm ...	103
9547	Ceviz Aacl	12	Ko uyolu Mahallesi, Muhittin st_nda Cadd...	77
9548	Huqqa	5	Kuru_e me Mahallesi, Muallim Naci Caddesi, N...	73
9549	Ak Kahve	11	Kuru_e me Mahallesi, Muallim Naci Caddesi, N...	75
9550	Walter's Coffee Roastery	24	Cafea Mahallesi, Bademaltl Sokak, No 21/B, ...	65

9551 rows × 4 columns

Create new features like "Has Table Booking" or "Has Online Delivery" by encoding categorical variables

```
In [41]: df["Has Table Booking"] = df["Has Table booking"].apply(lambda x: 1 if x == "Yes" else 0)
df["Has Online Delivery"] = df["Has Online delivery"].apply(lambda x: 1 if x == "Yes" else 0)

In [42]: df[["Has Table booking", "Has Table Booking", "Has Online delivery", "Has Online Delivery"]]
```

Out[42]:

	Has Table booking	Has Table Booking	Has Online delivery	Has Online Delivery
0	Yes	1	No	0
1	Yes	1	No	0
2	Yes	1	No	0
3	No	0	No	0
4	Yes	1	No	0
...	...	...	...	...
9546	No	0	No	0
9547	No	0	No	0
9548	No	0	No	0
9549	No	0	No	0
9550	No	0	No	0

9551 rows × 4 columns

Build a regression model to predict the aggregate rating of a restaurant based on available features. Split the dataset into training and testing sets and evaluate the model's performance using appropriate metrics.

```
In [43]: from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
In [44]: df = pd.get_dummies(df, columns = ["Has Table booking", "Has Online delivery"], drop_
```

```
In [45]: x = df[["Average Cost for two", "Votes", "Price range", "Has Table booking_Yes", "Has
y = df["Aggregate rating"]
```

```
In [46]: x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=
```

```
In [47]: model = LinearRegression()
model.fit(x_train, y_train)

y_pred = model.predict(x_test)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Model :Linear Regression")
print("Mean Square Error (MSE) :", mse)
print("R-squared (R2) Score", r2)
```

```
Model :Linear Regression
Mean Square Error (MSE) : 1.6764802747031446
R-squared (R2) Score 0.26344464090219477
```

Experiment with different algorithms(eg:linear regression, decision trees, random forest) and compare their performance

```
In [48]: from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
```

```
In [49]: models = {
    "Linear Regression" : LinearRegression(),
    "Decision Tree" : DecisionTreeRegressor(random_state=42),
    "Random Forest" : RandomForestRegressor(random_state=42)
}

results = {}
for name, model in models.items():
    model.fit(x_train, y_train)
    y_pred = model.predict(x_test)
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    results[name] = {"MSE" : mse, "R2 Score" : r2}

results_df = pd.DataFrame(results)
print(results_df)
```

	Linear Regression	Decision Tree	Random Forest
MSE	1.676480	0.203498	0.133938
R2 Score	0.263445	0.910594	0.941155

Analyze the relationship between the type of cuisine and the restaurant's rating

```
In [50]: cuisines= df["Cuisines"]
```

```
In [51]: cuisines.value_counts().head(10)
```

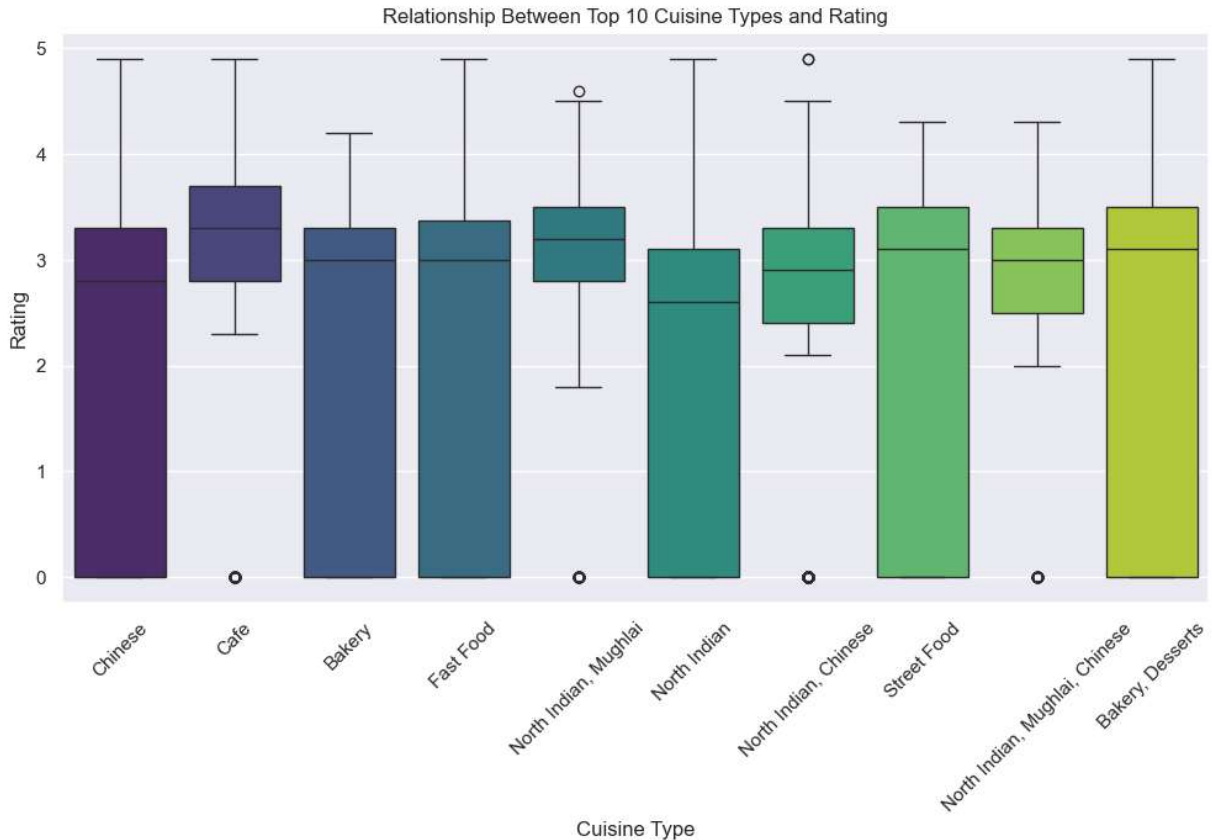
```
Out[51]: Cuisines
North Indian          936
North Indian, Chinese  511
Fast Food             354
Chinese               354
North Indian, Mughlai  334
Cafe                  299
Bakery                218
North Indian, Mughlai, Chinese  197
Bakery, Desserts      170
Street Food           149
Name: count, dtype: int64
```

```
In [52]: top_10_cuisines = cuisines.value_counts().head(10).index
```

```
In [53]: cuisine_ratings = pd.DataFrame({"cuisine" : cuisines, "Rating" : df["Aggregate rating"]})
```

```
In [54]: cuisine_ratings_top_10 = cuisine_ratings[cuisine_ratings["cuisine"].isin(top_10_cui
```

```
In [55]: plt.figure(figsize=(12,6))
sns.boxplot(x="cuisine",y="Rating", data=cuisine_ratings_top_10, palette = "viridis")
plt.title("Relationship Between Top 10 Cuisine Types and Rating")
plt.xlabel("Cuisine Type")
plt.ylabel("Rating")
plt.xticks(rotation=45)
plt.show()
```



Identify the most popular cuisines among customers based on the number of votes.

```
In [56]: cuisine_votes = pd.DataFrame({"Cuisine" :cuisines, "Votes" : df["Votes"]})
```

```
In [57]: cuisine_votes_sum = cuisine_votes.groupby("Cuisine")["Votes"].sum()
```

```
In [58]: popular_cuisines = cuisine_votes_sum.sort_values(ascending=False)
```

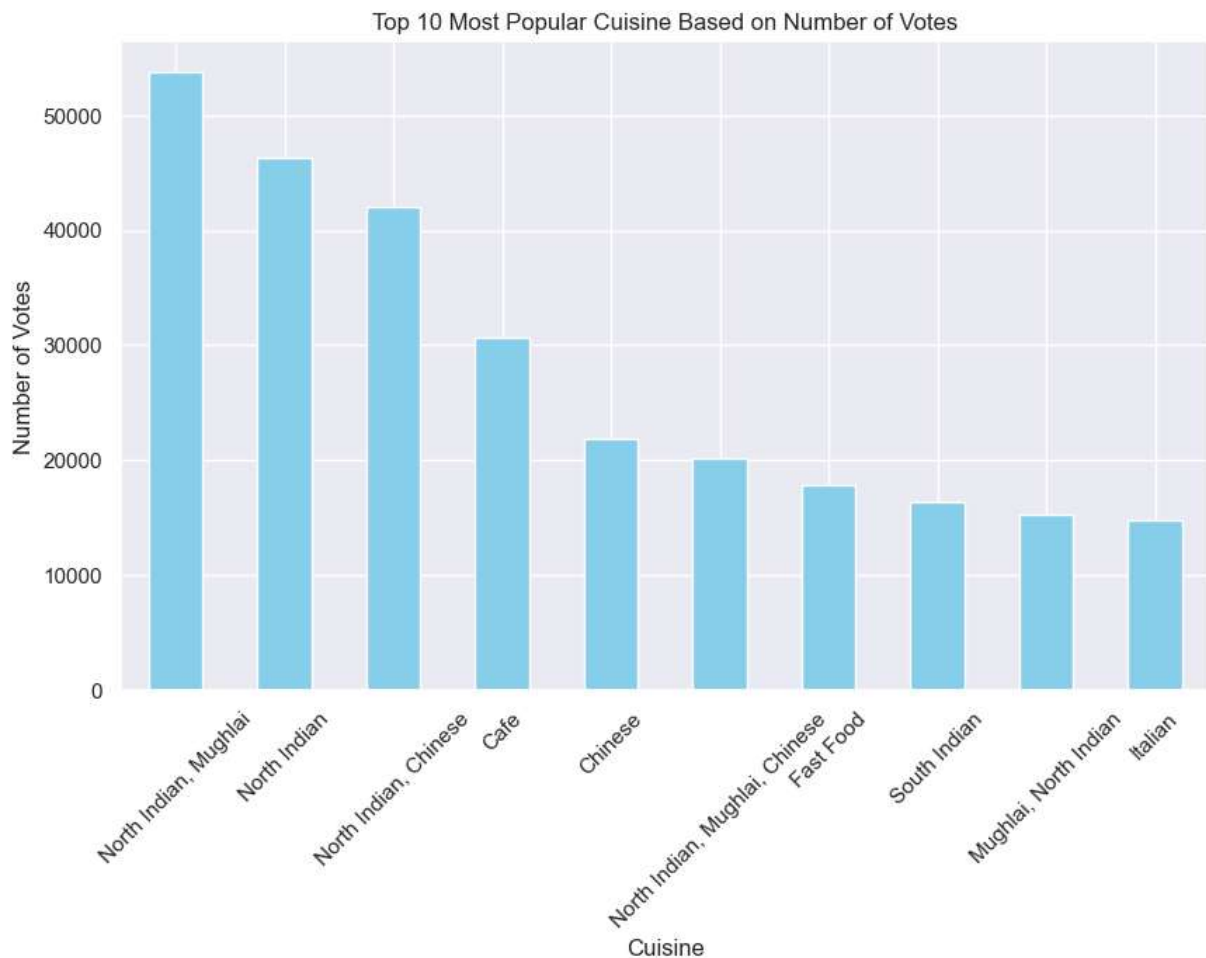
```
In [59]: print("Top 10 Most Popular Cuisines Based on Number of Votes :")
print(popular_cuisines.head(10))
```

Top 10 Most Popular Cuisines Based on Number of Votes :

Cuisine	Votes
North Indian, Mughlai	53747
North Indian	46241
North Indian, Chinese	42012
Cafe	30657
Chinese	21925
North Indian, Mughlai, Chinese	20115
Fast Food	17852
South Indian	16433
Mughlai, North Indian	15275
Italian	14799

Name: Votes, dtype: int64

```
In [60]: plt.figure(figsize=(10,6))
popular_cuisines.head(10).plot(kind="bar",color="skyblue")
plt.title("Top 10 Most Popular Cuisine Based on Number of Votes")
plt.xlabel("Cuisine")
plt.ylabel("Number of Votes")
plt.xticks(rotation=45)
plt.show()
```



Determine if there are any specific cuisines that tend to receive higher ratings.

```
In [61]: cuisine_ratings = pd.DataFrame({"Cuisine":cuisines,"Rating":df["Aggregate rating"]})

In [62]: average_rating_by_cuisine = cuisine_ratings.groupby("Cuisine")["Rating"].mean()

In [63]: sorted_cuisines_by_rating = average_rating_by_cuisine.sort_values(ascending=False)

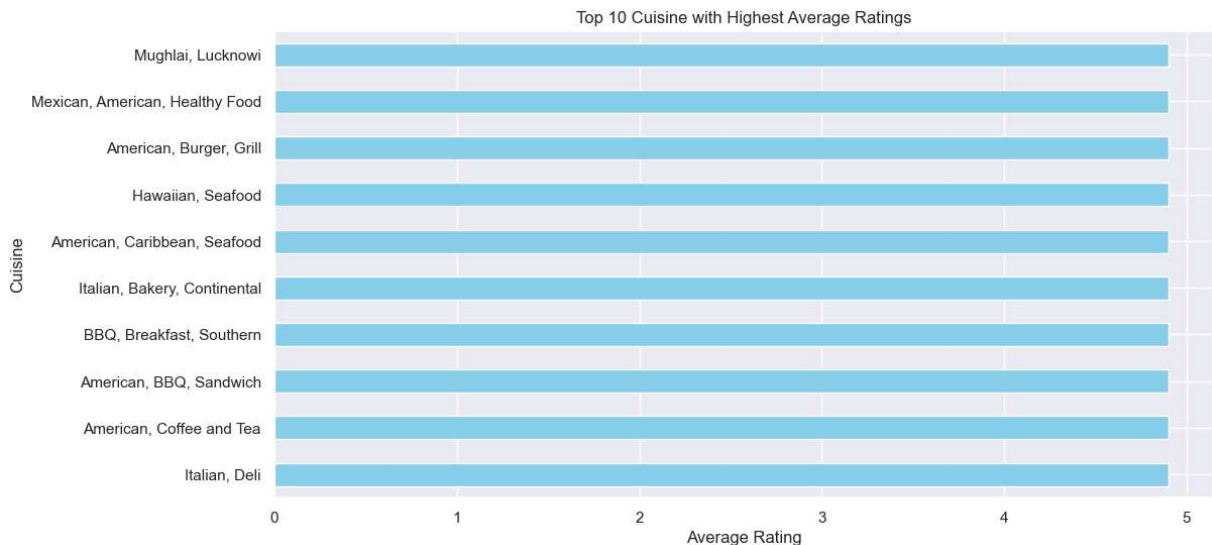
In [64]: print("Top 10 Cuisines with the Highest Average Ratings: ")
print(sorted_cuisines_by_rating.head(10))
```

Top 10 Cuisines with the Highest Average Ratings:

Cuisine	Rating
Italian, Deli	4.9
American, Coffee and Tea	4.9
American, BBQ, Sandwich	4.9
BBQ, Breakfast, Southern	4.9
Italian, Bakery, Continental	4.9
American, Caribbean, Seafood	4.9
Hawaiian, Seafood	4.9
American, Burger, Grill	4.9
Mexican, American, Healthy Food	4.9
Mughlai, Lucknowi	4.9

Name: Rating, dtype: float64

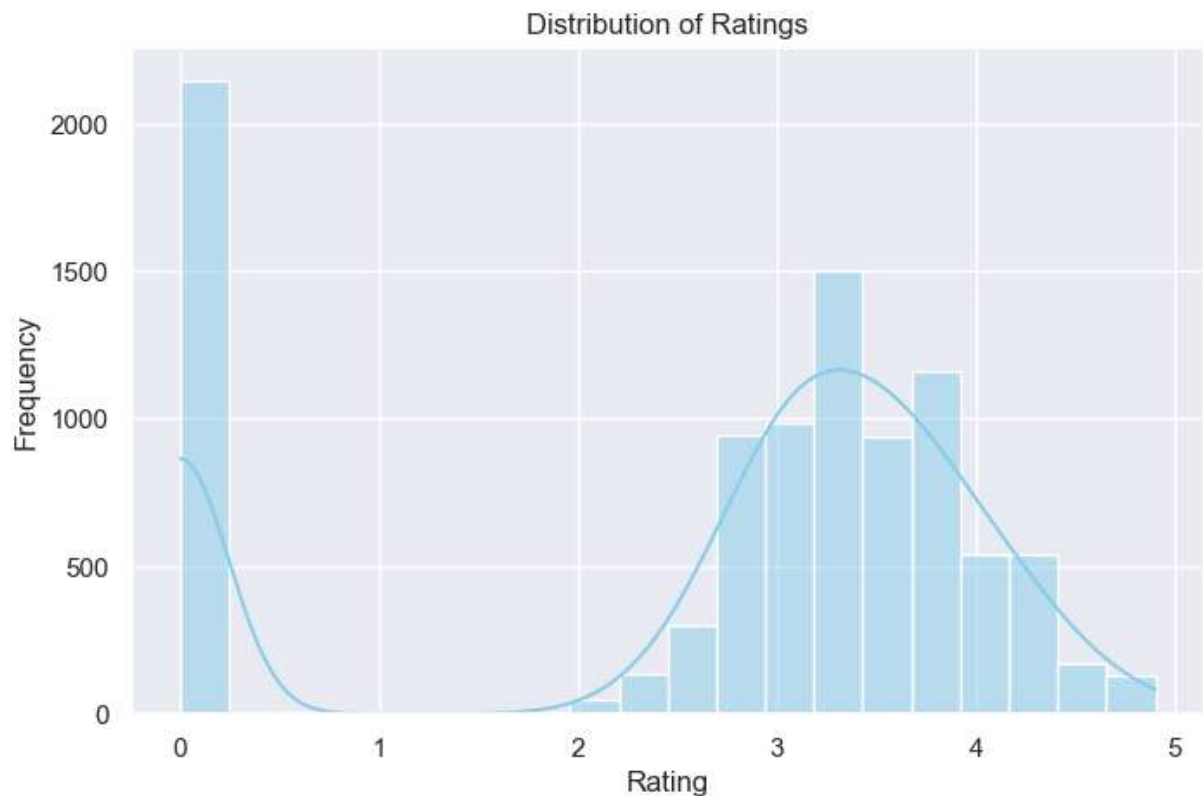
```
In [65]: plt.figure(figsize=(12,6))
sorted_cuisines_by_rating.head(10).plot(kind="barh",color="skyblue")
plt.title("Top 10 Cuisine with Highest Average Ratings")
plt.xlabel("Average Rating")
plt.ylabel("Cuisine")
plt.show()
```



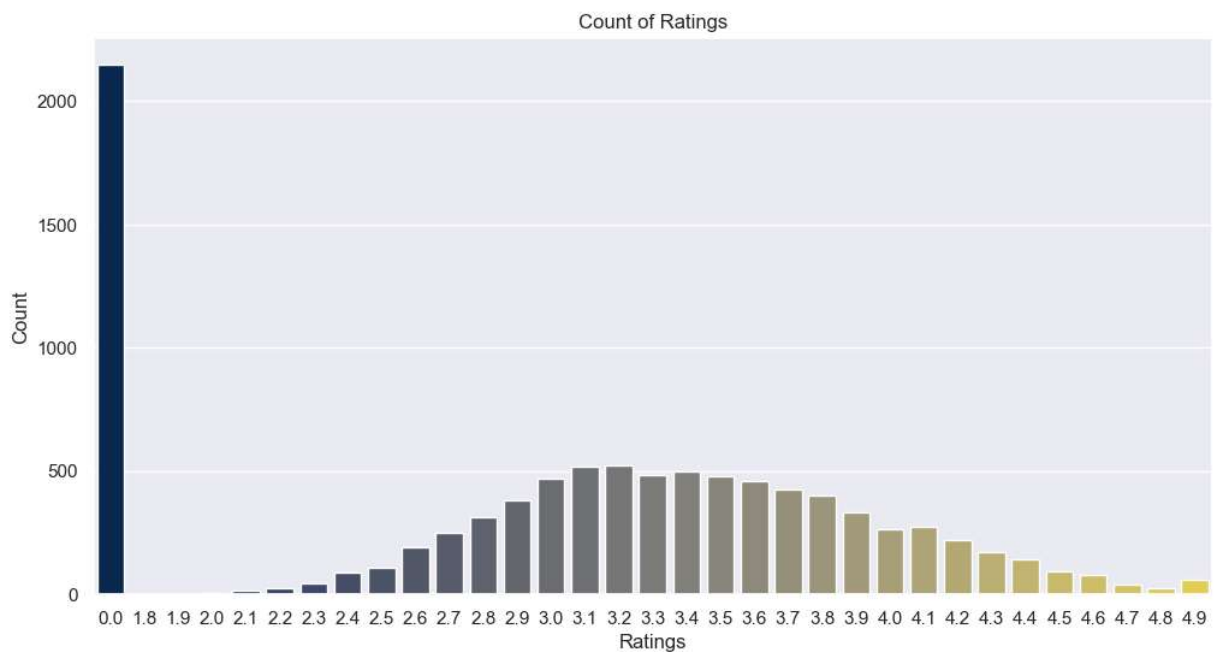
Create Visualizations to represent the distribution of rating using different charts(histogram,bar plot,etc).

```
In [66]: plt.figure(figsize=(8,5))
sns.histplot(df["Aggregate rating"],bins=20,kde=True,color="skyblue")
plt.title("Distribution of Ratings")
plt.xlabel("Rating")
```

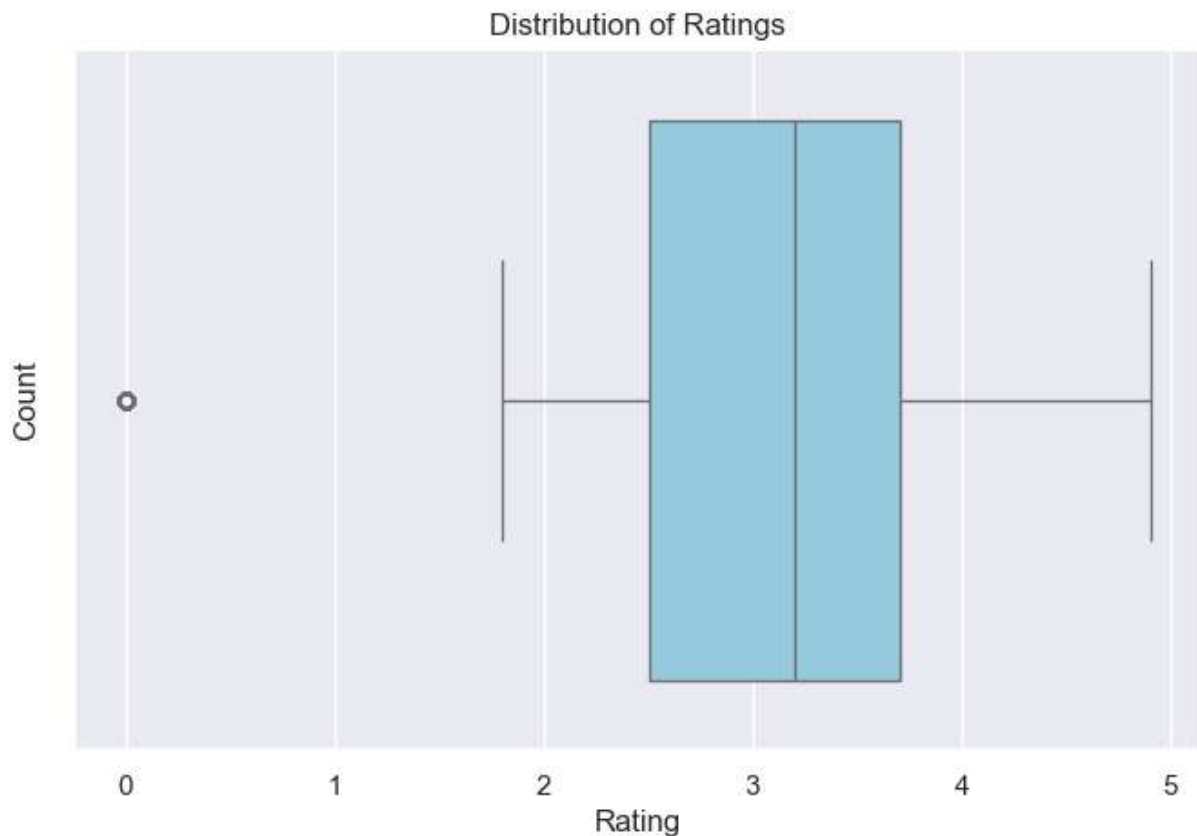
```
plt.ylabel("Frequency")
plt.show()
```



```
In [67]: plt.figure(figsize=(12,6))
sns.countplot(x="Aggregate rating",data = df, palette = "cividis")
plt.title("Count of Ratings")
plt.xlabel("Ratings")
plt.ylabel("Count")
plt.show()
```



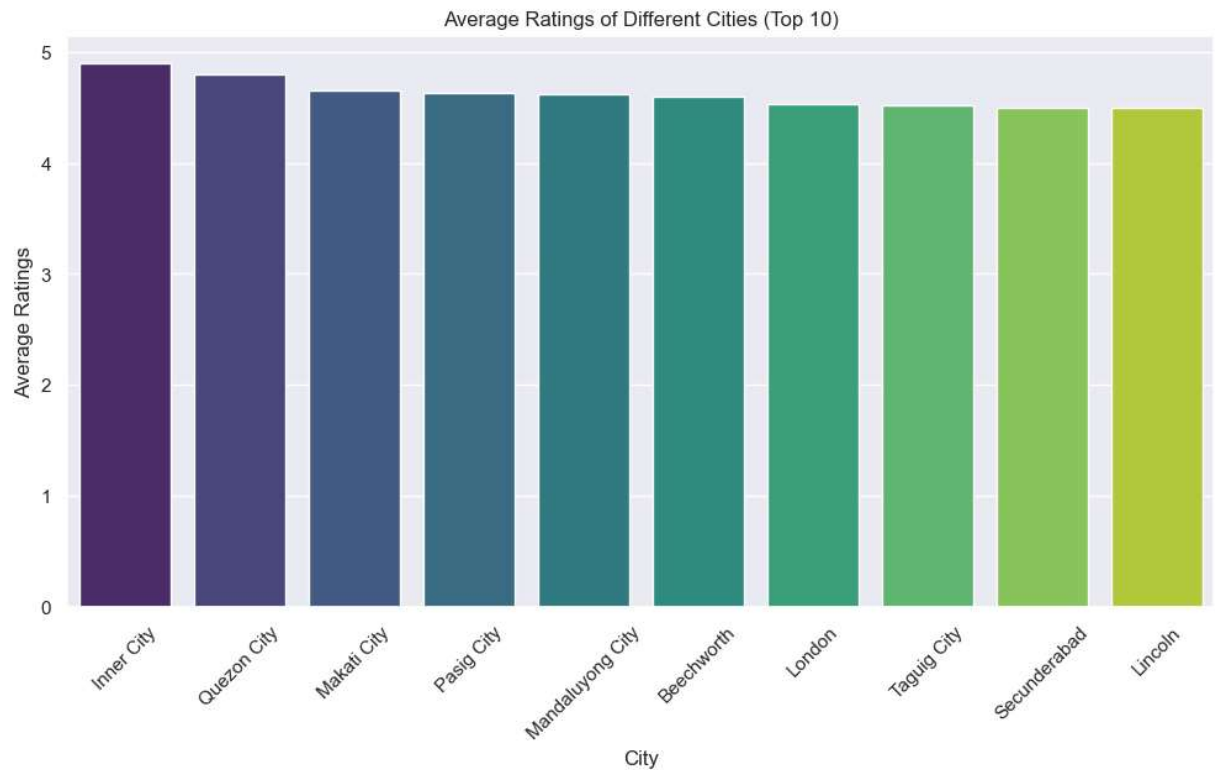
```
In [68]: plt.figure(figsize=(8,5))
sns.boxplot(x="Aggregate rating",data=df,color="skyblue")
plt.title("Distribution of Ratings")
plt.xlabel("Rating")
plt.ylabel("Count")
plt.show()
```



Compare the average ratings of different cuisines or cities using appropriate visualizations.

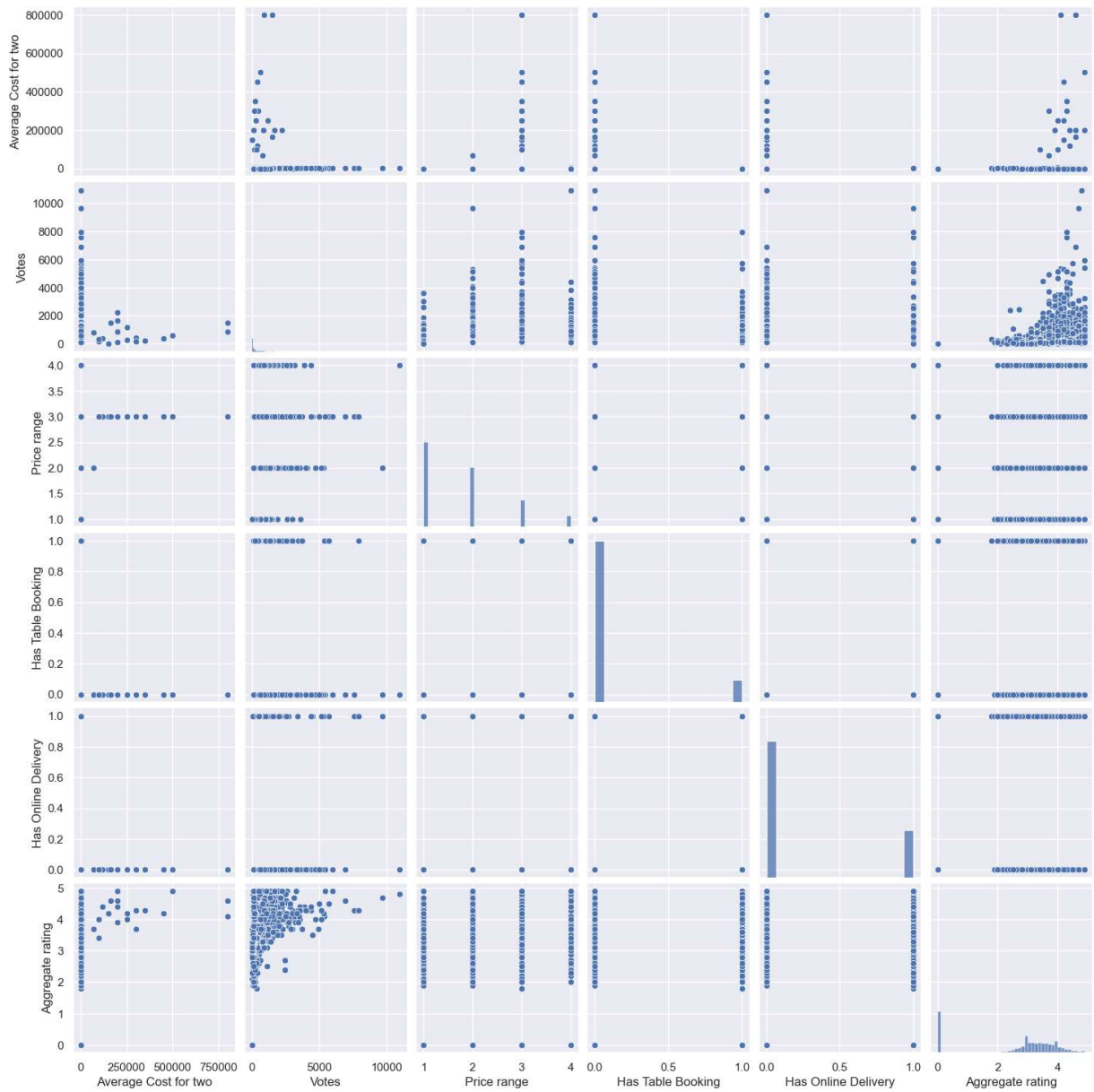
```
In [69]: average_rating_by_city = df.groupby("City")["Aggregate rating"].mean().sort_values(
```

```
In [70]: plt.figure(figsize=(12,6))
sns.barplot(x=average_rating_by_city.head(10).index,y=average_rating_by_city.head(10))
plt.title("Average Ratings of Different Cities (Top 10)")
plt.xlabel("City")
plt.ylabel("Average Ratings")
plt.xticks(rotation=45)
plt.show()
```



Visualize the relationship between various features and the target variable to gain insights.

```
In [73]: features = ["Average Cost for two", "Votes", "Price range", "Has Table Booking", "Has O  
sns.pairplot(df[features])  
plt.show()
```



In []: